OVERVIEW

液态玻璃，是一条可以顺着读懂的渲染链。

它由五步组成：先确定背景，再裁出组件所在区域，建立圆角光学场，计算折射与材质，最后把结果保存为稳定画面。文字和交互始终留在普通 DOM 上层。



READING RULE

从背景到快照是一条数据链；从 Canvas 到文字是一层合成关系。

01 / BACKGROUND SAMPLING

裁出来的，必须是背景里的同一块。

左图是铺满视口的背景，虚线框标出玻璃组件的位置。右图只把框内的原始纹理等比放大；此时还没有折射，也没有圆角遮罩。

1. VIEWPORT / COVER



2. LOCAL TEXTURE



同一纹理，等比放大

虚线只标示采样范围，不会写入背景纹理。圆角形状由下一步 Shader 的遮罩决定。

1 MEASURE

读取组件的 rect：left、top、width、height。

2 CROP

使用同一 cover 比例，按 rect 换算局部偏移。

3 UPLOAD

将局部 Canvas 上传为该组件的背景纹理。

LOCAL CROP FROM THE CONTROLLED BACKGROUND SOURCE

```
const rect = card.getBoundingClientRect();
const scale = Math.max(vw / iw, vh / ih);
const dx = (vw - iw * scale) / 2 - rect.left;
const dy = (vh - ih * scale) / 2 - rect.top;
ctx.drawImage(image, dx, dy, iw * scale, ih * scale);
```

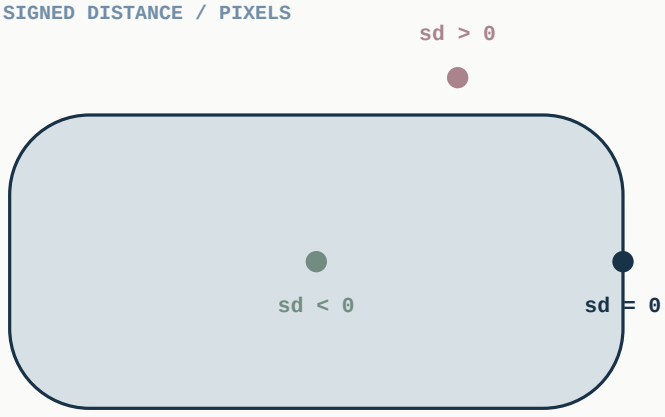
这里重建的是受控背景源与主题蒙层，并非给整个页面截图。普通 DOM、视频和另一层玻璃不会自动进入纹理。

02 / ROUNDED GEOMETRY

距离边界多远，与走到边界几成。

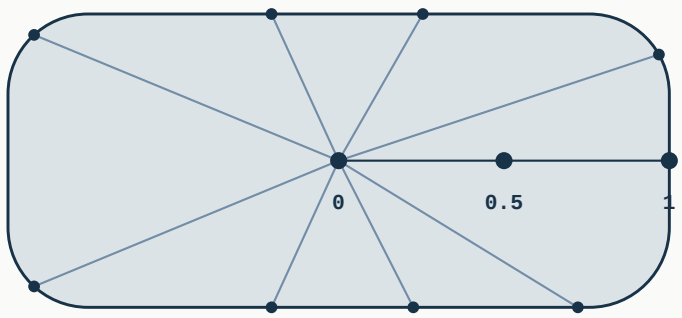
SDF 给出到边界的有符号距离；径向归一化给出从中心走向该方向边界的比例。两者用途不同，不能当成同一个量。

A. SIGNED DISTANCE / PIXELS



内部为负，边界为 0，外部为正。
用于形状遮罩，以及按像素控制边缘厚度。

B. RADIAL FRACTION / UNITLESS



每条射线都止于真实边界。
中心为 0，该方向的半程为 0.5，边界为 1。

ROUNDED RECTANGLE SDF

```
vec2 q = abs(point-u_res*.5)
  -(u_res*.5-vec2(u_corner));
float sd = length(max(q,0.))
  + min(max(q.x,q.y),0.) - u_corner;
```

NORMALIZED RADIAL DISTANCE

```
vec2 rel = px - u_res * .5;
vec2 p = rel / radialBoundary(rel);
float s = length(p);
// static shape: center 0, boundary 1
```

左图决定轮廓，右图组织整张玻璃的光学变化。

例如同样向内 5 个像素，适合用 SDF 描述；从中心走到边界的一半，适合用径向比例描述。后者不是到最近边界的距离。

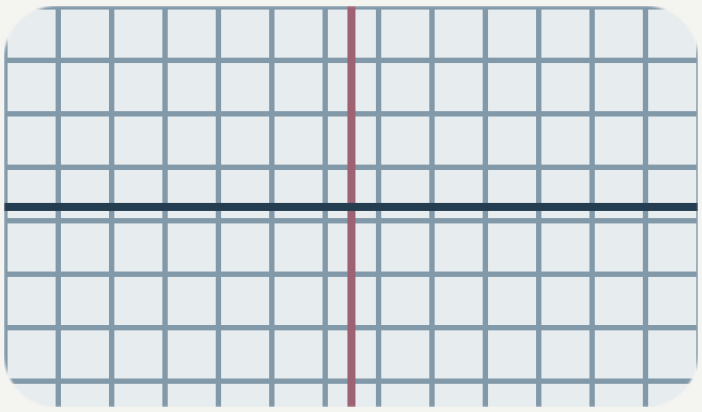
图示为静态几何。生产 Shader 在此基础上加入轻微的边界波动，再得到用于折射的 radius 与 rr；不能把图中的比例 s 直接等同于动画中的 rr。

03 / REFRACTION FIELD

折射，本质上是换一个位置读取背景。

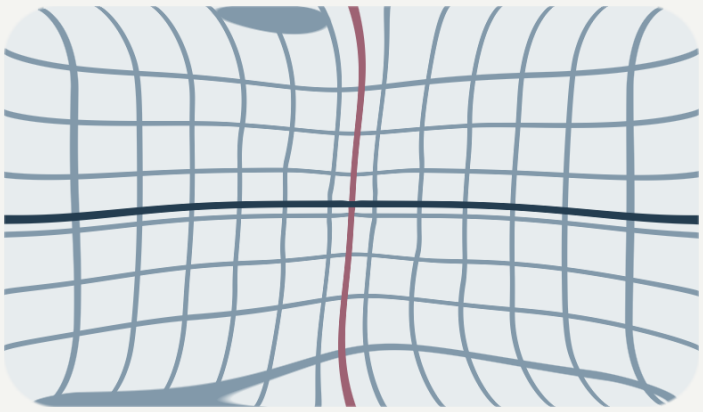
每个像素先得到归一化位置 p，再由整体透镜和低频波动计算位移 shift。最终不是给像素涂颜色，而是去背景纹理的邻近位置重新取色。

1. SOURCE GRID / ORIGINAL UV



SHIFT UV

2. SAME GRID / REFRACTED UV



网格按生产位移公式重算，固定 t = 0，仅展示折射采样项；没有添加烟灰、色调或高光。完整材质的实机截图见下一页。

p
归一化位置

rr / ang
半径与方向

shift
透镜与低频波动

refrUv
新的采样坐标

CONDENSED FROM THE PRODUCTION SHADER

```
float edgeGain = smoothstep(0.05, 0.97, rr);
float strength = (0.052 + 0.12 * edgeGain)
  * u_refract * u_lDistort;
vec2 shift = p * sh + interiorWave * strength;
vec2 refrUv = v_uv + vec2(shift.x/u_res.x, -shift.y/u_res.y);
vec3 refr = backdrop(refrUv, u_blur * u_lBlur);
```

CURRENT TUNING

26	0	0	25	13	15
REFRACT	TINT	BLUR	EDGE	SHINE	WOBBLE

这些是 Shader 调节刻度，不是物理单位。

04 / MATERIAL AND LIGHT

同一算法，两种玻璃。

下面是手机浏览器的真实渲染截图。两块玻璃使用相同的背景、裁切和尺寸；图中已经包含折射、材质与边缘光学效果。

01 / CLEAR



透明：背景透射较亮。树枝和窗框在玻璃内弯曲，上层文字仍保持清晰。

02 / SMOKED



烟灰：主体透射降低，仍能看见背景细节；内侧暗边与局部高光保留体积感。

这是两套生产材质预设的对照。它们共用同一套光学算法，但烟灰预设还将折射强度乘以 1.06、高光强度乘以 0.92，并使用不同的色调与边缘参数。

LUMINANCE ADAPTATION / SHADER EXCERPT

SAMPLED BRIGHTNESS CONTROLS THE SMOKED EDGE

```
float smokeLum = dot(color, vec3(0.2126, 0.7152, 0.0722));
float smokeBright = smoothstep(0.42, 0.84, smokeLum);
float smokeWidth = mix(7.2, 4.25, smokeBright);
float smokeEdgeLoss = mix(0.27, 0.085, smokeBright);
```

边缘如何适应背景

亮区：吸光边缘更窄、更轻；暗区：保留更多吸光厚度。判断依据是采样颜色的亮度，而不是日间或夜间标签。

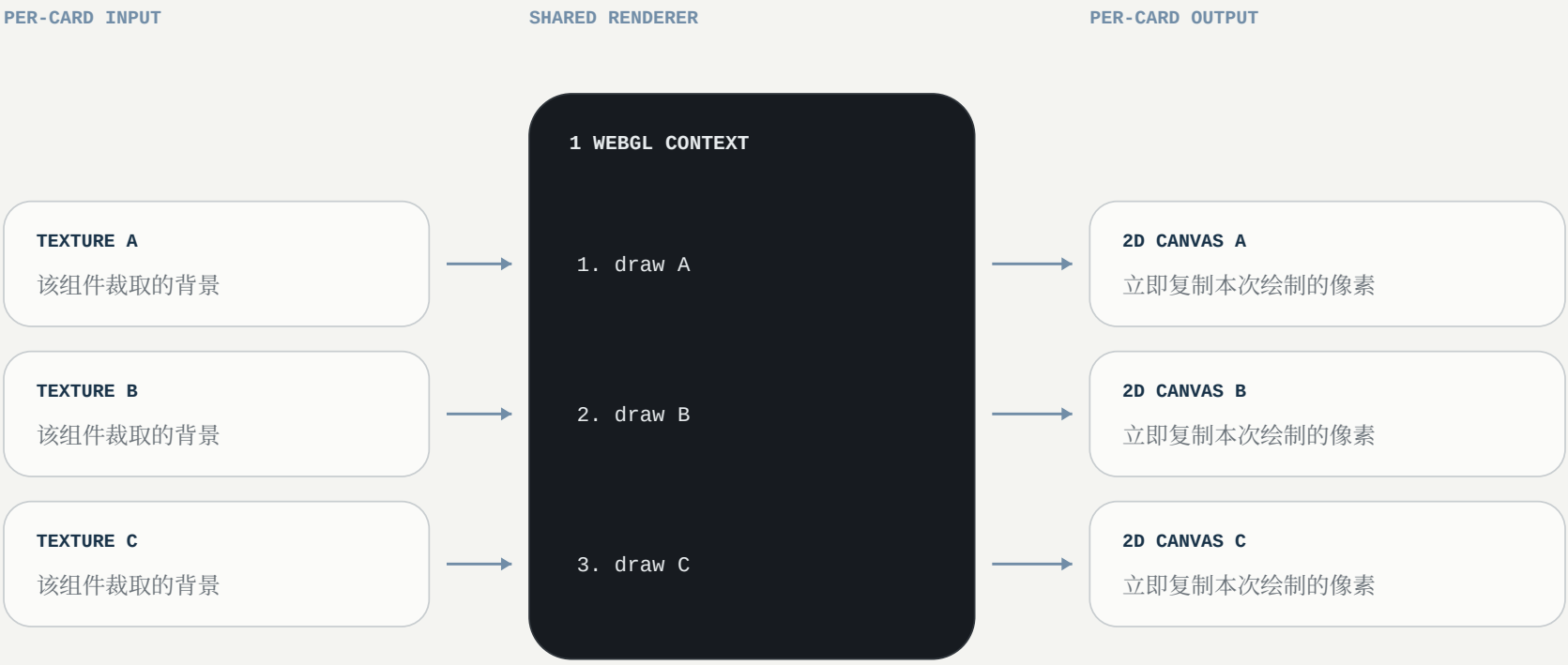
PHONE CAPTURE / 2026-11-05 / PRODUCTION SHADER V5

截图以静态图片嵌入，PDF 无需运行 WebGL。图像仅裁切排版，未重绘或添加玻璃效果。

05 / SHARED RENDERER

轮流借用渲染器，各自保存画面。

每个组件有自己的背景纹理与输出 Canvas。它们依次借用同一个 WebGL 绘制表面：绘制 A 后立刻复制给 A，再处理 B。



三个编号表示先后顺序，不是三个并行的 GPU 渲染器。文字与交互仍位于输出 Canvas 上方的 DOM 中。

DRAW, COPY IMMEDIATELY, THEN PROCESS THE NEXT CARD

```
gl.drawArrays(gl.TRIANGLE_STRIP, 0, 4);
// Copy this card before another card reuses the surface.
output.clearRect(0, 0, width, height);
output.drawImage(renderSurface, 0, 0);
```

OFFSCREEN / THREE DIFFERENT RESOURCES

GPU TEXTURE	BACKDROP CANVAS	OUTPUT 2D CANVAS
删除纹理 重新进入时重建	采样画布尺寸归零 释放这份像素缓冲	保留最近画面 GPU 丢失时仍可显示

“1 个 context”指单个文档中的一次玻璃引擎实例。独立 iframe 各有自己的实例，截图对照页因此使用两个。

06 / LIFECYCLE AND REUSE

离屏会暂停，回来还能继续。

可见性决定是否绘制；dirty 表示背景纹理需要重建；节点移除才销毁组件。这三件事不能画成一条单向流程。

DIRTY / INDEPENDENT FLAG

壁纸、主题、滚动或尺寸变化使背景纹理失效。下次绘制前重建；尺寸变化可能立即触发绘制。

VISIBLE

观察区内：允许绘制

页面隐藏时暂停

减少动态效果时仅按需重绘

离开观察区 →

← 重新进入

OFFSCREEN

观察区外：暂停绘制

释放纹理与背景缓冲

保留自己的 2D 输出快照

节点移除（任一状态）

DISPOSE

断开监听、释放纹理、移除输出画布，并从渲染集合中注销。

FRAME POLICY, CONDENSED

```
01    if (disposed || !visible || document.hidden || w < 5) return;
02    if (reduced) { if (dirty || generationChanged) draw(0); return; }
03    if (ms - last < 48) return;   // visible-card frame cap
04    last = ms; draw(ms * .001);  // texture rebuilt only when dirty
```

CURRENT BUDGETS

1.5	~ 20	40 px	2.25 px	CSS
DPR CAP	VISIBLE FPS	IO MARGIN	HERO BLUR	FALLBACK

PUBLIC SCOPE

只发布光学说明、选定的示例图与实现片段。完整应用、聊天后端与账户数据不在其中。上游来源与许可说明见仓库 CREDITS.md。

PROJECT: github.com/yanceydaisy/liquid-glass-note